```
import csv

def load_and_count_dataset(file_path, keywords):
    """
    Parses the CSV file using pure Python and counts total reviews,
    positive reviews, and keyword occurrences.
    """
    total_reviews = 0
    total_positive = 0
    keyword_counts = {kw: 0 for kw in keywords}
    keyword_positive_counts = {kw: 0 for kw in keywords}

    with open(file_path, mode='r', encoding='utf-8') as f:
        reader = csv.reader(f)
        header = next(reader)  # Discard the header row

        for row in reader:
            if len(row) < 2:
                continue
            review = row[0].lower()
            sentiment = row[1].lower()

            total_reviews += 1
            is_positive = (sentiment == 'positive')
            if is_positive:
                total_positive += 1

            for kw in keywords:
                if kw in review:
                    keyword_counts[kw] += 1
                    if is_positive:
                        keyword_positive_counts[kw] += 1

    return total_reviews, total_positive, keyword_counts, keyword_positive_counts


def compute_bayes_probabilities(total_reviews, total_positive, keyword_counts, keyword_positive_counts):
    """
    Applies Bayes' Theorem to compute Prior, Likelihood, Marginal, and Posterior.
    """
    prior_pos = total_positive / total_reviews
    results = {}

    for kw in keyword_counts:
        # Avoid zero division errors if a keyword never occurs
        if keyword_counts[kw] == 0:
            results[kw] = (prior_pos, 0.0, 0.0, 0.0)
            continue

        likelihood = keyword_positive_counts[kw] / total_positive
        marginal = keyword_counts[kw] / total_reviews

        # Bayes' Theorem formula implementation
        posterior = (likelihood * prior_pos) / marginal

        results[kw] = {
            "Prior P(Positive)": prior_pos,
            "Likelihood P(keyword|Positive)": likelihood,
            "Marginal P(keyword)": marginal,
            "Posterior P(Positive|keyword)": posterior
        }
    return results


def display_results(results):
    """
    Prints a clean, presentation-ready output for each keyword.
    """
    for kw, metrics in results.items():
        print(f"\n========================================")
        print(f"KEYWORD: '{kw.upper()}'")
        print(f"========================================")
        for metric_name, value in metrics.items():
            print(f"{metric_name:<32}: {value:.4f}")
```

```
# Main execution flow following the DRY principle
DATASET_PATH = "IMDB Dataset.csv"

positive_keywords = ['wonderful', 'excellent', 'amazing', 'masterpiece']
negative_keywords = ['waste', 'awful', 'worst', 'boring']
target_keywords = positive_keywords + negative_keywords

# 1. Extract frequencies
total_n, total_pos, counts, positive_counts = load_and_count_dataset(DATASET_PATH, target_keywords)

# 2. Apply Bayes' Theorem
bayes_metrics = compute_bayes_probabilities(total_n, total_pos, counts, positive_counts)

# 3. Present outputs
display_results(bayes_metrics)
```

```
========================================
KEYWORD: 'WONDERFUL'
========================================
Prior P(Positive)               : 0.4942
Likelihood P(keyword|Positive)  : 0.1112
Marginal P(keyword)             : 0.0673
Posterior P(Positive|keyword)   : 0.8170

========================================
KEYWORD: 'EXCELLENT'
========================================
Prior P(Positive)               : 0.4942
Likelihood P(keyword|Positive)  : 0.1194
Marginal P(keyword)             : 0.0728
Posterior P(Positive|keyword)   : 0.8107

========================================
KEYWORD: 'AMAZING'
========================================
Prior P(Positive)               : 0.4942
Likelihood P(keyword|Positive)  : 0.0777
Marginal P(keyword)             : 0.0506
Posterior P(Positive|keyword)   : 0.7588

========================================
KEYWORD: 'MASTERPIECE'
========================================
Prior P(Positive)               : 0.4942
Likelihood P(keyword|Positive)  : 0.0364
Marginal P(keyword)             : 0.0252
Posterior P(Positive|keyword)   : 0.7140

========================================
KEYWORD: 'WASTE'
========================================
Prior P(Positive)               : 0.4942
Likelihood P(keyword|Positive)  : 0.0126
Marginal P(keyword)             : 0.0724
Posterior P(Positive|keyword)   : 0.0858

========================================
KEYWORD: 'AWFUL'
========================================
Prior P(Positive)               : 0.4942
Likelihood P(keyword|Positive)  : 0.0129
Marginal P(keyword)             : 0.0618
Posterior P(Positive|keyword)   : 0.1030

========================================
KEYWORD: 'WORST'
========================================
Prior P(Positive)               : 0.4942
Likelihood P(keyword|Positive)  : 0.0166
Marginal P(keyword)             : 0.0867
Posterior P(Positive|keyword)   : 0.0946

========================================
```